

Databases 2 - exam - January 22, 2026 - Dur. 2h

S. Comai, P. Fraternali, D. Martinenghi

A. Ranking (10 points, 2 points per question)

With the Milano-Cortina 2026 Winter Olympics approaching, fans are comparing the best skiers throughout history. The following table D reports World Cup victories by discipline (Slalom (SL), Giant Slalom (GS), Super-G (SG), Downhill (DH), and Alpine Combined (AC)) for top male skiers.

Skier Name	SL	GS	SG	DH	AC
MG: Marc Girardelli	16	7	9	3	11
MH: Marcel Hirscher	32	31	1	0	3
HM: Hermann Maier	0	14	24	15	1
BM: Bode Miller	5	9	5	8	6
MO: Marco Odermatt	0	28	16	6	0
IS: Ingemar Stenmark	40	46	0	0	0
AS: Aksel Lund Svindal	0	4	17	14	1
GT: Gustav Thöni	8	11	0	0	5
AT: Alberto Tomba	35	15	0	0	0
PZ: Pirmin Zurbriggen	2	7	10	10	11

- The skyline is known to identify the potentially optimal items of a dataset, but in which sense is an item potentially optimal? A tuple t is potentially optimal in dataset D if [check all that apply]
 - ☐ $\exists f \in L_1 \forall s \in D (f(t) \geq f(s))$, where L_1 is the set of linear norms
 - ☐ $\exists f \forall s \in D (s \neq t \rightarrow f(t) > f(s))$
 - ☐ $\exists f \in \mathcal{M} \forall s \in D (s \neq t \rightarrow f(t) > f(s))$, where $\mathcal{M}$ is the set of monotone functions
 - ☐ $\exists f \in L_1 \forall s \in D (s \neq t \rightarrow f(t) > f(s))$
- Compute the skyline of the dataset D shown in the table. Just give the result (use initials of skier names) and list just **two** dominance relationships that hold in D .
- Let D' be the same as D but without the columns **SL** and **DH**. What is the skyline of D' ?
- Let D'' be the same as D' but without the column **SG**. What is the skyline of D'' ?
- Compare the skylines of D , D' , D'' and state whether, in this case and in general, removing columns from a dataset increases/reduces its skyline.

Solution

- The only correct answer is the third (potential optimality is with respect to a monotone function and no ties).
- The skyline of D is $\{MG, MH, HM, BM, MO, IS, GT, PZ\}$. In other words, the only skiers not in the skyline are AS (dominated by HM) and AT (dominated by IS). So two dominance relationships are $HM < AS$ and $IS < AT$ (where $<$ stands for dominance).
- The skyline of D' is $\{MH, HM, BM, MO, IS, GT, PZ\}$. In other words, MG exits the skyline, as it is now dominated by PZ.
- The skyline of D'' is $\{MG, MH, BM, IS, GT, PZ\}$. In other words, MG re-enters the skyline, as it now coincides with (and thus is no longer dominated by) PZ; HM and MO exit the skyline, as they are dominated by MH.
- The skylines of D' and D'' are strict subsets of the skyline of D . However, the skyline of D'' is not a subset of that of D' , because a dominance relationship in D' ($PZ < MG$) disappears in D'' , even though D'' is obtained by removing a column from D' . In conclusion, although removing columns makes it virtually more “difficult” for a tuple to be in the skyline, there is no guarantee on how the skyline will vary (so its size might increase, decrease or stay the same).

B. Concurrency control (12 points)

Consider the following schedule of transactions:

$S = r_1(y), w_2(x), r_1(x), r_1(z), w_1(x), r_2(z), r_3(x), w_1(z), w_3(y)$

- Classify S with respect to VSR (in case, listing the view-equivalent schedules), CSR, 2PL, Strict 2PL, and TS Multi (with the conventions adopted for TS Multi under Snapshot Isolation, used for the exercises). Justify your answers exhaustively. Use class inclusion whenever possible. For 2PL and TS multi show the result using the table. (10 points, 2 points for each class)
- Suppose that transaction T2 aborts immediately after executing $r_2(z)$. Explain why this abort may compromise the transaction isolation property in the schedule. Which concurrency control mechanism can prevent this situation? Explain briefly. (2 points)

Solution.

- Short answers:

CSR: Yes (2PL implies CSR)

VSR: Yes (CSR implies VSR). View-equivalent to: T2, T1, T3.

2PL: Yes

Strict 2PL: No, T2 must release x before $r_1(x)$ but can commit only after $r_2(z)$

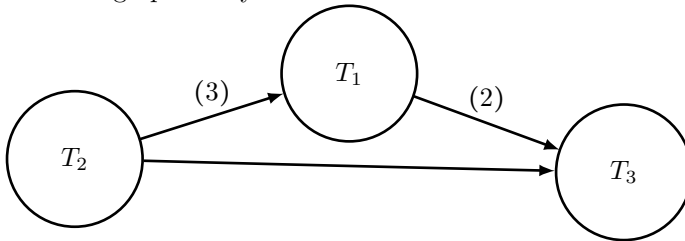
TS multi: No, because of $w_2(x) - w_1(x)$

- If T2 aborts after $r_2(z)$, T1 has already read the value x written by T2 (an uncommitted value) and therefore should be aborted as well. A dirty read has occurred.

Under strict two-phase locking (strict 2PL), read operations are allowed only on committed data, preventing dirty reads; in fact, the given schedule cannot be generated by a strict 2PL protocol ($S \notin \text{Strict2PL}$).

Detailed analysis for the CSR class

The conflict graph is acyclic.



Detailed analysis for the VSR class

The schedule is CSR and thus also VSR.

View-equivalent to serial schedule: $w_2(x), r_2(z), r_1(y), r_1(x), r_1(z), w_1(x), w_1(z), r_3(x), w_3(y)$

Detailed analysis for the 2PL class

	1	2	3	4	5	6	7	8	9
x	$\uparrow_2 w_2$	$\downarrow_2$	$\uparrow_1 r_1$	$\uparrow_1 w_1$			$\downarrow_1$	$\uparrow_3 r_3$	$\downarrow_3$
y	$\uparrow_1 r_1$						$\downarrow_1$	$\uparrow_3$	$w_3 \downarrow_3$
z		$\uparrow_2$		$\uparrow_1 r_1$	$r_2 \downarrow_2$	$\uparrow_1$			$w_1 \downarrow_1$
		2				1			3

Legend

$\uparrow$: read lock request
 $\uparrow$: write lock request
 $\uparrow$: lock upgrade
 $\downarrow$: unlock request

Detailed analysis for the Strict 2PL class

T2 must release x before r1(x) but can commit only after r2(z).

	1	2	3	4	5	6	7	8	9
x		w_2	$\downarrow_2$	r_1		w_1		$\downarrow_1$	r_3
y	r_1								w_3
z				r_1		r_2	$\bullet^2$		w_1
								$\bullet^1$	

Legend

$\downarrow$: mandatory unlock request

$\bullet$: commit time

Detailed analysis for the TS Multi class

The schedule is not TS Multi because there are killed transactions.

Operation	RTM(x)	WTM(x)	RTM(y)	WTM(y)	RTM(z)	WTM(z)	Killed transactions
$r_1(y)$	0 (0)	0	1 (0)	0	0 (0)	0	
$w_2(x)$		0, 2					
$r_1(x)$	1 (0)						
$r_1(z)$					1 (0)		
$w_1(x)$							1 as 1 < last WTM(x) = 2
$r_2(z)$					2 (0)		
$r_3(x)$	3 (2)						
$w_1(z)$							1 (already killed)
$w_3(y)$				0, 3			

C. JPA (10 points)

An academic application manages the data of authors and publications. The database schema comprises the following tables:

Author(id, name, email, country)

AuthorProfile(author_id, orcid, institution, bio)

Article(article_id, title, publicationDate, main_author_id)

Keyword(keyword_id, name)

Article_keywords(article_id, keyword_id)

Each author has optionally one AuthorProfile and a profile belongs to exactly one author. Each article has exactly one main author (the corresponding author) and an author can be the main author of several articles. Other non corresponding authors are not represented in the the specified database. An article can be tagged with many keywords and several articles can use the same keyword. The database will contain approximately 50.000 authors, up to 50.000 profile instances, 200.000 article instances and 2.000 keyword instances. An author typically is the main author of 1 to 50 articles, an article has on average 5 to 10 keywords and popular keywords may be associated with tens of thousands of articles. The application supports the following access patterns: 1) given an author's id, load the author, its profile and the author's articles. 2) Given a keyword name, find the articles tagged with that keyword and for each article load its main author. 3) Given an article Id, load the article with its main author and all associated keywords.

With regard to the above relational schema:

a) Fill in the tables on the next page for the indicated relationships/directions. In the Annotation field, report the complete JPA annotation (e.g., starting with **@ManyToMany**), including all relevant attributes (e.g., **mappedBy**, **fetch**, **cascade**). If 'Required?' is 'No', still specify all the other mapping properties in the table, as if the relationship were mapped. Always choose an owner. (6 points)

b) Write the code of the JPA entity that maps the **Article** entity (and its attributes). Take into account the above-mentioned access paths of the application and the cardinalities. Explicitly specify the owner of the relationships, the chosen fetch policies and cascade types: for each aspect, justify and discuss your design choices. (4 points)

(The space below can be used for the solution)

Solution.

```
@Entity
public class Article {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long article_id;
    private String title;
    @Temporal(TemporalType.DATE)
    private Date publicationDate;

    // N:1 to Author (main author)
    // Needed for pattern #3
    // No cascading needed, multiple articles may have the same main author
    @ManyToOne(fetch = FetchType.EAGER)
    @JoinColumn(name = "main_author_id")
    private Author mainAuthor;

    // M:N Article <-> Keyword (owning side)
    // Needed for pattern #3
    // No cascading, M:N relation
    @ManyToMany(
        fetch = FetchType.EAGER, // small collection
    )
    @JoinTable(
        name = "article_keywords",
        joinColumns = @JoinColumn(name = "article_id"),
        inverseJoinColumns = @JoinColumn(name = "keyword_id")
    )
    private List<Keyword> keywords;

    // getters and setters omitted for brevity
}
```

Relation I: Author → AuthorProfile		
	<i>Answer</i>	<i>Motivate your answer</i>
Required?	Yes	Needed for pattern #1
Owner	AuthorProfile	FK authorId is on AuthorProfile, making it owner of the 1:1 relation
FetchType	EAGER	The default is ok, only 1 correlated instance
CascadeType	REMOVE	If author is deleted also the profile is deleted
Annotation	<pre>@OneToOne(mappedBy = "author", fetch = FetchType.EAGER, cascade = CascadeType.REMOVE) private AuthorProfile profile;</pre>	

Relation II: AuthorProfile → Author		
	<i>Answer</i>	<i>Motivate your answer</i>
Required?	No	Can be mapped bidirectionally for symmetry
Owner	AuthorProfile	FK authorId is on AuthorProfile, making it owner of the 1:1 relation
FetchType	EAGER	Only 1 correlated entity
CascadeType	None	Operations are typically applied to author and cascaded to the profile
Annotation	<pre>@OneToOne(fetch = FetchType.EAGER) @JoinColumn(name = "author_id") private Author author;</pre>	

Relation III: Keyword → Article		
	<i>Answer</i>	<i>Motivate your answer</i>
Required?	YES	Needed for pattern #2
Owner	Article or Keyword	We exemplify Article as the owner (Keyword as owner is ok too)
FetchType	LAZY	Large collection
CascadeType	None	No cascading needed, N:M relation
Annotation	<pre>@ManyToMany(mappedBy = "keywords", fetch = FetchType.LAZY) private List<Article> articles;</pre>	

Relation IV: Author → Article		
	<i>Answer</i>	<i>Motivate your answer</i>
Required?	Yes	Needed for pattern #1
Owner	Article	1:N the FK is contained in the article table
FetchType	LAZY	mid-size collection, could be EAGER as well
CascadeType	REMOVE	deletion of main author implies articles are deleted too
Annotation	<pre>@OneToMany(mappedBy = "mainAuthor", fetch = FetchType.LAZY, cascade = CascadeType.REMOVE) private List<Article> articles;</pre>	